1. **Implement file handling operations:**
    1. **Create, open file and count chars, spaces, tabs, and new lines in a file.**
    2. **Append one file at the end of other.**
    3. **Capitalize the first letter of every word in a file**.

```
#include <stdio.h>
int main() {
   FILE *fp;
   char filename[] = "du.txt";
   char ch;
   int chars = 0, spaces = 0, tabs = 0, newlines = 0;
   fp = fopen(filename, "r");
   ch = fgetc(fp);
   while (ch != EOF) {
      if (ch == '\t')
         tabs++;
      else if (ch == ' ')
         spaces++;
      else if (ch == '\n')
         newlines++;
      else
            chars++;
      ch=fgetc(fp);
   }
   fclose(fp);
   printf("Character count: %d\n", chars);
   printf("Spaces: %d\n", spaces);
   printf("Tabs: %d\n", tabs);
   printf("New lines: %d\n", newlines);

   return 0;
}

#include <stdio.h>
int main() {
   FILE *fp1, *fp2;
   char file1[100], file2[100];
   char ch;
   printf("Enter the first file (destination): ");
   scanf("%s", file1);
   printf("Enter the second file (source): ");
   scanf("%s", file2);

   fp1 = fopen(file1, "a");
   fp2 = fopen(file2, "r");
   while ((ch = fgetc(fp2)) != EOF) {
```

```
        fputc(ch, fp1);
    }
    fclose(fp1);
    fclose(fp2);

    return 0;
}

#include <stdio.h>
#include <ctype.h>
int main() {
    FILE *fp_in, *fp_out;
    char inFile[100], outFile[] = "temp.txt";
    int ch, prev = ' ';

    printf("Enter the file name: ");
    scanf("%s", inFile);

    fp_in = fopen(inFile, "r");
    fp_out = fopen(outFile, "w");

    while ((ch = fgetc(fp_in)) != EOF) {
        if ((prev == ' ' || prev == '\n' || prev == '\t') && isalpha(ch)) {
            ch = toupper(ch);
        }
        fputc(ch, fp_out);
        prev = ch;
    }
    fclose(fp_in);
    fclose(fp_out);
    printf("Capitalization done successfully.\n");
    return 0;
}
```

2. Write a program to implement lexical analyzer.

```
#include <stdbool.h>
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
// Returns 'true' if the character is a DELIMITER.
bool isDelimiter(char ch)
{
	if (ch == ' ' || ch == '+' || ch == '-' || ch == '*' ||
		ch == '/' || ch == ',' || ch == ';' || ch == '>' ||
		ch == '<' || ch == '=' || ch == '(' || ch == ')' ||
		ch == '[' || ch == ']' || ch == '{' || ch == '}')
		return (true);
	return (false);
}
// Returns 'true' if the character is an OPERATOR.
bool isOperator(char ch)
{
	if (ch == '+' || ch == '-' || ch == '*' ||
		ch == '/' || ch == '>' || ch == '<' ||
		ch == '=')
		return (true);
	return (false);
}
// Returns 'true' if the string is a VALID IDENTIFIER.
bool validIdentifier(char* str)
{
	if (str[0] == '0' || str[0] == '1' || str[0] == '2' ||
		str[0] == '3' || str[0] == '4' || str[0] == '5' ||
		str[0] == '6' || str[0] == '7' || str[0] == '8' ||
		str[0] == '9' || isDelimiter(str[0]) == true)
		return (false);
	return (true);
}
// Returns 'true' if the string is a KEYWORD.
bool isKeyword(char* str)
{
	if (!strcmp(str, "if") || !strcmp(str, "else") ||
		!strcmp(str, "while") || !strcmp(str, "do") ||
		!strcmp(str, "break") ||
		!strcmp(str, "continue") || !strcmp(str, "int")
		|| !strcmp(str, "double") || !strcmp(str, "float")
		|| !strcmp(str, "return") || !strcmp(str, "char")
		|| !strcmp(str, "case") || !strcmp(str, "char")
		|| !strcmp(str, "sizeof") || !strcmp(str, "long")
		|| !strcmp(str, "short") || !strcmp(str, "typedef")
```

```
                || !strcmp(str, "switch") || !strcmp(str, "unsigned")
                || !strcmp(str, "void") || !strcmp(str, "static")
                || !strcmp(str, "struct") || !strcmp(str, "goto"))
                return (true);
        return (false);
}
// Returns 'true' if the string is an INTEGER.
bool isInteger(char* str)
{
        int i, len = strlen(str);

        if (len == 0)
                return (false);
        for (i = 0; i < len; i++) {
                if (str[i] != '0' && str[i] != '1' && str[i] != '2'
                        && str[i] != '3' && str[i] != '4' && str[i] != '5'
                        && str[i] != '6' && str[i] != '7' && str[i] != '8'
                        && str[i] != '9' || (str[i] == '-' && i > 0))
                        return (false);
        }
        return (true);
}
// Returns 'true' if the string is a REAL NUMBER.
bool isRealNumber(char* str)
{
        int i, len = strlen(str);
        bool hasDecimal = false;

        if (len == 0)
                return (false);
        for (i = 0; i < len; i++) {
                if (str[i] != '0' && str[i] != '1' && str[i] != '2'
                        && str[i] != '3' && str[i] != '4' && str[i] != '5'
                        && str[i] != '6' && str[i] != '7' && str[i] != '8'
                        && str[i] != '9' && str[i] != '.' ||
                        (str[i] == '-' && i > 0))
                        return (false);
                if (str[i] == '.')
                        hasDecimal = true;
        }
        return (hasDecimal);
}
// Extracts the SUBSTRING.
char* subString(char* str, int left, int right)
{
        int i;
```

```
        char* subStr = (char*)malloc(
                                sizeof(char) * (right - left + 2));

        for (i = left; i <= right; i++)
                subStr[i - left] = str[i];
        subStr[right - left + 1] = '\0';
        return (subStr);
}
// Parsing the input STRING.
void parse(char* str)
{
        int left = 0, right = 0;
        int len = strlen(str);

        while (right <= len && left <= right) {
                if (isDelimiter(str[right]) == false)
                        right++;

                if (isDelimiter(str[right]) == true && left == right) {
                        if (isOperator(str[right]) == true)
                                printf("'%c' IS AN OPERATOR\n", str[right]);

                        right++;
                        left = right;
                } else if (isDelimiter(str[right]) == true && left != right
                                || (right == len && left != right)) {
                        char* subStr = subString(str, left, right - 1);

                        if (isKeyword(subStr) == true)
                                printf("'%s' IS A KEYWORD\n", subStr);

                        else if (isInteger(subStr) == true)
                                printf("'%s' IS AN INTEGER\n", subStr);

                        else if (isRealNumber(subStr) == true)
                                printf("'%s' IS A REAL NUMBER\n", subStr);

                        else if (validIdentifier(subStr) == true
                                        && isDelimiter(str[right - 1]) == false)
                                printf("'%s' IS A VALID IDENTIFIER\n", subStr);

                        else if (validIdentifier(subStr) == false
                                        && isDelimiter(str[right - 1]) == false)
                                printf("'%s' IS NOT A VALID IDENTIFIER\n", subStr);
                        left = right;
                }
```

```
        }
        return;
}
// DRIVER FUNCTION
int main()
{
        // maximum length of string is 100 here
        char str[100] ;
        printf("enter string:");
        scanf("%s",str);
        parse(str); // calling the parse function

        return (0);
}
```

3. Implement following programs using LEX.

1. Write a LEX program to count number of vowels and consonants in each input string.

2. Write a LEX program to take input from text file and count no of characters, no. of lines & no. of words

3. Write a LEX program to identify whether a given line is a comment or not.

4. Implement all the task done by lexical analyzer.

1. Number of vowels and consonants:

```
%option noyywrap
%{
	#include<stdio.h>
	int vowels=0;
	int consonants=0;
%}
%%
[aeiouAEIOU] {vowels++;}
[a-zA-Z] {consonants++;}
"\n" {printf("%d\n", vowels); vowels = 0;}
%%
int main()
{
	printf("Enter the string:\n");
	yylex();
	return 0;
}
```

2. Count no of char, lines and words:

```
%option noyywrap
%{
#include<stdio.h>
int c=0,l=0,w=0;
%}
%%
[a-zA-Z0-9]* {w++;c+=yyleng;}
. {c++;}
\n {c++;l++;}
%%
void main()
{
	yyin=fopen("input6.txt","r");
	yylex();
	printf("No of charachters: %d\nNo of words: %d\nNo of lines: %d\n",c,w,l);
}
```

3. Find comment lines:

```
%option noyywrap
%{
#include<stdio.h>
int a=1;
%}
%%
\/\/.*\n {printf("%d. %s\n",a,yytext);a++;}
\/\*(.*\n)*.*\*\/ { printf("%d. %s\n",a,yytext);a++;}
.|\n {}
%%
void main()
{
        Printf("No of comment lines are: %d",a);
        yylex();
}
```

4. Lexical analyzer:

```
%option noyywrap
%{
#include <stdio.h>
int
keyword=0,ids=0,posint=0,posfloat=0,negint=0,negfloat=0,opertor=0,space=0,linebk=0,cmt=0,specialsy
m=0;
%}

%%
\/\*(.*\n)*.*\*\/                                   {cmt++;}
"//".*\n                                    {cmt++;}
\.|~|@|#|\$|\(|\)|_|,|\?|\'|\"|;|:|\[|\]|\{|\}  {specialsym++;   printf("\nSp.   Sym.   %s   \t   Length=
%d",yytext,yyleng);}
auto|double|int|struct|break|else|long|switch|case|enum|register|typedef|char|extern|return|unio
n|const|float|short|unsigned|continue|for|signed|void|default|goto|sizeof|volatile|do|if|static|whil
e        {keyword++; printf("\nKeyword %s \t Length=   %d",yytext,yyleng);}
[A-Za-z_][A-Za-z0-9_]+                      {ids++; printf("\nIdentifier %s \t Length=   %d",yytext,yyleng);}
\+?[0-9]+                                   {posint++;printf("\nInteger %s \t Length=   %d",yytext,yyleng);}
\+?[0-9]+"."[0-9]+                          {posfloat++;printf("\nFloat %s \t Length=   %d",yytext,yyleng);}
-[0-9]+                                     {negint++;printf("\n-ve Int %s \t Length=   %d",yytext,yyleng);}
-[0-9]+"."[0-9]+                            {negfloat++;printf("\n-ve        Float        %s        \t        Length=
%d",yytext,yyleng);}
\+|\-|\*|\/|\^|\>|\>=|\<|\<=|\=|\==|\!=|&|\|%        {opertor++;printf("\nOperator   %s   \t   Length=
%d",yytext,yyleng);}
[ \t]                                       {space++;}
[\n]                                        {linebk++;}
```

```
%%
void main(int argc, char **argv)
{
        FILE *fp;
        int val;
        if((fp = fopen(argv[1],"r"))==NULL)
        yyin=stdin;
        else
        yyin = fp;

        yylex();
        printf("\nKeywords:- %d",keyword);
        printf("\nIDs:- %d",ids);
        printf("\n+ve int:- %d",posint);
        printf("\n+ve float:- %d",posfloat);
        printf("\n-ve int:- %d",negint);
        printf("\n-ve float:- %d",negfloat);
        printf("\nOperators:- %d",opertor);
        printf("\nSpecial Symbols:- %d",specialsym);
        printf("\nSpaces:- %d",space);
        printf("\nComments:- %d",cmt);
        printf("\nLine breaks:- %d\n",linebk);
}
```

4. Write a program to implement DFA for (a+b)*abb

```
#include<stdio.h>
#include<string.h>
void main()
{
	char arr[20],a;
	int l,i=0,c=0;
	printf("enter string:");
	scanf("%s",arr);
	l=strlen(arr);
	while(arr[i]!=NULL)
	{
		if(arr[i]=='a'||arr[i]=='b')
		{
			c++;
		}
		else
		{
			printf("not valid string");
			break;
		}
		i++;
	}
	if(c>0)
	{
		if((arr[l-1]=='b')&&(arr[l-2]=='b')&&(arr[l-3]=='a'))
			printf("The string is accepted");
		else
			printf("The string is not accepted");
	}
}
```

5. Write a program to check given input string can be derived by following grammar or not.

S → Aa | bAc | dc | bda

A → d

```
#include <stdio.h>
#include <string.h>
int check_grammar(const char *str) {
   // Check for rule S -> Aa (A -> d), so string should be "da"
   if (strcmp(str, "da") == 0) {
      return 1;
   }
   // Check for rule S -> bAc (A -> d), so string should be "bdc"
   if (strcmp(str, "bdc") == 0) {
      return 1;
   }
   // Check for rule S -> dc
   if (strcmp(str, "dc") == 0) {
      return 1;
   }
   // Check for rule S -> bda
   if (strcmp(str, "bda") == 0) {
      return 1;
   }
   return 0;
}

int main() {
   char input[100];
   printf("Enter a string: ");
   scanf("%s", input);

   if (check_grammar(input)) {
      printf("The string can be derived from the grammar.\n");
   } else {
      printf("The string cannot be derived from the grammar.\n");
   }

   return 0;
}
```

6. 1. Write a program to eliminate left recursion from a grammar production.
2. Write a program to perform left factoring on grammar production.

1. Left Recursion

```
#include<stdio.h>
#include<string.h>
int main()
{
	int len=0,i,j,k;
	char str[100],a1[100]="\0",a2[100]="\0";
	printf("Kindly put / between production and = after left side nonterminal");
	printf("\n\tEnter The Expression : ");
	scanf("%s",str);

	len = strlen(str);
	for(i=2;i<len;i++)
	{
		if(str[i]==str[0] && (str[i-1] =='/' || str[i-1] == '='))
		{
			printf("\n\n\tEntered Expression is left Recursive");
			break;
		}
	}

	if(i==len)
	{
		printf("\n\n\tEntered Expressin is not left Recursive");
		getch();
		return 0;
	}

	else
	{
		a1[0] = str[0];
		a1[1] = str[1];
		j=2;
		for(i=2;i<len;i++)
		{
			if(str[i]!=str[0] && (str[i-1] =='/' || str[i-1] == '='))
			{
				for(k=i;k<len;k++)
				{
					a1[j]=str[k];
					j++;
if(str[k]=='/')
						break;
```

```
                        }
                        if(a1[j-1]!='/')
                        {
                                a1[j]=a1[0];
                                a1[j+1]='\'';
                                continue;
                        }
                        else
                                a1[j-1]=a1[0];
                        a1[j]='\'';
                        a1[j+1]='/';
                        j+=2;
                        i=k;
                }
        }
        a2[0] = a1[0];
        strcat(a2, "'=&/");
        j=strlen(a2);
        for(i=2;i<len;i++)
        {
                if(str[i]==str[0] && (str[i-1] =='/' || str[i-1] == '='))
                {
                        for(k=i+1;k<len;k++)
                        {
                                a2[j]=str[k];
                                j++;
                                if(str[k]=='/')
                                break;
                        }
                        if(a2[j-1]!='/')
                        {
                                a2[j]=a2[0];
                                a2[j+1]='\'';
                                continue;
                        }
                        else
                                a2[j-1]=a1[0];
                        a2[j]='\'';
                        a2[j+1]='/';
j+=2;
                        i=k;
                }
        }

        if(a1[strlen(a1)-1]=='/')
                a1[strlen(a1)-1]='\0';
```

```
        if(a2[strlen(a2)-1]=='/')
                a2[strlen(a2)-1]='\0';

        printf("\n\n\t%s\n\n\t%s",a1,a2);

    }

    return 0;
}
```

2. Left Factoring

```
#include<stdio.h>
#include<string.h>
int main()
{
    int len=0,i,j,k,l,t=0,flag=0,ck[20];
    char str[100],t1[20][10]={"\0"},a1[100]="\0",a2[10][100]={"\0"},com[20]="\0";
    printf("Kindly put / between production and = after left side nonterminal");
    printf("\n\tEnter The Expression : ");
    scanf("%s",str);
    len = strlen(str);
    for(i=2;i<len;i++)
    {
        if(str[i-1]=='='||str[i-1]=='/')
        {
            for(l=i+1;l<len;l++)
            {
                if(str[l]==str[i] && (str[l-1] =='/' || str[l-1] == '='))
                {
                    printf("\n\n\ left factoring is possible");
                    flag=1;
                    break;
                }
            }
                if(flag==1)
                    break;
        }
    }
    if(flag==0)
    {
        printf("\n\n\ left factoring is not possible");
        getch();
        return 0;
    }
    else
```

```
        {
                for(i=2;i<len;i++)
                {
                        if(str[i-1] =='/' || str[i-1] == '=')
                        {
                                j=0;
                                for(l=i;l<len;l++)
                                {
                                        t1[t][j]=str[l];
                                        if(str[l+1]=='/'||str[l+1]=='\0')
                                                break;
                                        j++;
                                }
                                t++;
                        }
                }

                for(i=0;;i++)
                {
                        for(j=0;j<20;j++)
                        {
                                if(com=="\0" && t1[j]!="\0")
                                        strcpy(com,t1[j]);
                                for(k=0;k<strlen(com);k++)
                                {

                                }
                        }
                }
        }


        return 0;
}
```

7. Write a program to implement recursive decent parser for following grammar:

E → E + T | T

T → T * F | F

F → id

```
#include<stdio.h>
#include<ctype.h>
#include<string.h>
#define TRUE 1
#define FALSE 0
#define SIZE 20
#define ID 2
#define NUM 3
#define PLUS 4
#define ASTERISK 5
short int E();
short int T();
short int Eprime();
short int Tprime();
short int F();
char string[SIZE];	/* the input string */
int i=0;	/* i for the input string */
short int gettoken();
void error();
short int E()
{
	short int found=FALSE;
	if(T())
		if(Eprime())
			found=TRUE;
	return found;
}
short int Eprime()
{
	short int found=FALSE;
	short int save=i;
	if(gettoken()==PLUS)
	{
if(T())
			if(Eprime())
				found=TRUE;
	}
	else
	{
		i=save;
		found=TRUE;
```

```
        }
        return found;
}
short int T()
{
        short int found=FALSE;
        if(F())
                if(Tprime())
                        found=TRUE;
        return found;
}
short int Tprime()
{
        short int found=FALSE;
        short int save=i;
        if(gettoken()==ASTERISK)
        {
                if(F())
                        if(Tprime())
                                found=TRUE;
        }
        else
        {
                i=save;
                found=TRUE;
        }
        return found;
}
short int F()
{
        short int found=FALSE;
        short int token=gettoken();
        if(token==ID || token==NUM)
                found=TRUE;
        return found;
}
short int gettoken()
{
        while(string[i]==' ')
                i++;
        if(isalpha(string[i]))
        {
                i++;
                while(string[i]!=0 && string[i]!='*' && string[i]!='+')
                {
                        if(!isalnum(string[i++]))
```

```
                        error();
                }
                return ID;
        }
        else if(isdigit(string[i]))
        {
                i++;
                while(string[i]!=0 && string[i]!='*' && string[i]!='+')
                {
                        if(!isdigit(string[i++]))
                        error();
                }
                return NUM;
        }
        else if(string[i]=='*')
        {
                i++;
                return ASTERISK;
        }
        else if(string[i]=='+')
        {
                i++;
                return PLUS;
        }
        else if(string[i]==0)
                return 0;
}
void error()
{
        printf("Lexical error.\n");
exit(0);
}

void main()
{
        printf("Enter the expression:\n");
        scanf("%s",string);
        if(E())
        {
                printf("Expression is syntactically correct.\n");
        }
        else
                printf("Syntax error.\n");

}
```

8. Write a program to find FIRST of given grammar.

```
#include<stdio.h>
#include<string.h>
#define no 10
void findf(char NT, char NTD, char QNT, char QNTD);
char expn[no][20] = {"\0"};
char ans[no][20] = {"\0"};
int size;
int main()
{
	int i,len=0;
	printf("How Many Grammer You Want To Enter ?? Max : %d >> ",no);
	scanf("%d",&size);
	printf("Enter The Grammer : \nEXAMPLE : E=TE' E'=+TE'/& T=FT' T'=*FT'/&		F=(E)/id");
	printf("\nNOTE: & = Epsilon And In Answer i = id \nEnter Your Grammer :");
	// E=TE' E'=+TE'/& T=FT' T'=*FT'/& F=(E)/id
	for(i=0; i<size; i++)
	{
		scanf("%s",expn[i]);
		if(expn[i][0]=='\0')
			break;
	}
	for(i=0; i<size; i++)
	{
		if(expn[i][0] == '\0')
			break;
		printf("\n%s\n", expn[i]);
	}

	for(i=0; i<size; i++)
	{
		if(expn[i][0] == '\0')
			break;
		ans[i][0] = expn[i][0];
		ans[i][1] = '=';
		ans[i][2] = '{';
		if(expn[i][1]=='\'')
		{
			ans[i][1] = expn[i][1];
			ans[i][2] = '=';
			ans[i][3] = '{';
		}
	}

	for(i=0; i<size; i++)
	{
```

```
                findf(ans[i][0], ans[i][1], ';', ';');
        }

        printf("\n\nFirst Of The Entered Grammer");
        for(i=0; i<size; i++)
        {
                if(ans[i][0] == '\0')
                        break;
                printf("\n\n%s", ans[i]);
        }

        return 0;
}
void findf(char NT, char NTD, char QNT, char QNTD)
{
        int i,j,k,len;
        for(i=0;i<size;i++)
        {
                if(ans[i][0] == NT && ans[i][1] == NTD)
                {
                        len = strlen(ans[i]);
                        if(ans[i][len-1]!='}')
                        {
                                for(j=2;j<strlen(expn[i]);j++)
                                {
                                        if((expn[i][j-1] =='/' || expn[i][j-1] == '='))
                                        {
                                                if(expn[i][j] < 65 || expn[i][j] > 90)
                                                {
                                                        ans[i][len] = expn[i][j];
                                                        len++;
                                                        ans[i][len] = ',';
                                                        len++;
                                                }
                                                else
                                                {
                                                        if(expn[i][j+1]=='\'')
                  findf(expn[i][j],'\'',ans[i][0],ans[i][1]);
                                                        else
                                                        findf(expn[i][j],'=',ans[i][0],ans[i][1]);
                                                }
                                        }

                                }
                                len = strlen(ans[i]);
                                if(ans[i][len-1]==',')
```

```
                        ans[i][len-1] = '}';
                else
                        ans[i][len] = '}';
        }
        for(j=0;j<size;j++)
        {
                if(QNT == ';' && QNTD == ';')
                        break;
                if(ans[j][0] == QNT && ans[j][1] == QNTD)
                {
                        len = strlen(ans[j]);
                        if(ans[i][2]=='{')
                                k=4;
                        else
                                k=5;
                        for(;k<strlen(ans[i]);k++)
                        {
                                ans[j][len] = ans[i][k-1];
                                len++;
                        }
                        break;
                }
        }
    }
  }
}
```

9. Write a program to find leading symbol of given production.

```
#include<stdio.h>
#include<string.h>
void main()
{
        char str[10];
        char temp,nt;
        int i,len;
        printf("Enter Nonterminal(In CAPITAL):");
        scanf("%c",&nt);
        printf("enter expression:");
        scanf("%s",str);
        len=strlen(str);
        for(i=0;i<=len;i++)
        {
        temp=str[i];
        if((temp>='a'&&temp<='z')||(temp=='+')||(temp=='-')||(temp=='*')||(temp=='/')||(temp=='^'))
                {
                  printf("leading for a given expression=%c", temp);
                  break;
                }
        }
}
```

10. Create YACC and LEX specification files to generate a calculator which accepts, integer and float type arguments.

Parse.y

```
%{
#include<stdio.h>
#include<stdlib.h>

int yylex();
int yyerror();
int result=0,flag=0;
%}
%union{
        double dv;
}
%token <dv> NUMBER
%type <dv> expr
%left '+' '-'
%left '*' '/'
%%
stmt :expr {printf("\n\nResult = %lf",$1);};
expr :expr'+'expr {$$=$1+$3;}
|expr'-'expr    {$$=$1-$3;}
|expr'*'expr    {$$=$1*$3;}
|expr'/'expr    {if($3==0)
  flag=1;
  else
  $$=$1/$3;}
|'('expr')' {$$=$2;}
|NUMBER {$$=$1;};
%%

int main()
{
printf("Enter the Arithmetic Expression : ");
if(flag!=1)
{
yyparse();
}
return 1;
}

int yyerror()
{
printf("Invalid Arithmetic Expression\n\n");
exit(0);
}
```

Parse.l

```
%{
#include"Parse.tab.h"
%}
```

```
%%
[0-9]+|[0-9]*\.[0-9]+ {
                yylval.dv= atof(yytext);
                return NUMBER;
                }
[ \t]+ {;}
. {return yytext[0];}
\n {return 0;}
%%

int yywrap()
{
return 1;
}
```

11. Write a program to convert expression from infix to postfix.

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>
#define MAX 100
char stack[MAX];
int top = -1;
/* push element into stack */
void push(char x)
{
stack[++top] = x;
}
/* pop element from stack */
char pop()
{
if(top == -1)
return -1;
else
return stack[top--];
}
/* check precedence of operators /
int precedence(char x)
{
if(x == '+' || x == '-')
return 1;
if(x == '' || x == '/')
return 2;
if(x == '^')
return 3;
return 0;
}
int main()
{
char infix[100], postfix[100];
int i, j = 0;
printf("Enter Infix Expression: ");
scanf("%s", infix);

for(i = 0; i < strlen(infix); i++)
{
   char ch = infix[i];

   /* if operand, add to postfix */
   if(isalnum(ch))
   {
      postfix[j++] = ch;
   }

   /* if '(', push to stack */
   else if(ch == '(')
   {
      push(ch);
```

```
    }

    /* if ')', pop until '(' */
    else if(ch == ')')
    {
        while(stack[top] != '(')
        {
            postfix[j++] = pop();
        }
        pop();
    }

    /* if operator */
    else
    {
        while(top != -1 && precedence(stack[top]) >= precedence(ch))
        {
            postfix[j++] = pop();
        }
        push(ch);
    }
}

/* pop remaining operators */
while(top != -1)
{
    postfix[j++] = pop();
}

postfix[j] = '\0';

printf("Postfix Expression: %s\n", postfix);

return 0;
}
```

12. Write a program to read input string from a file and generate three address code.

```
#include <stdio.h>
#include <string.h>
int main()
{
FILE *fp;
char input[100];
int i = 0;
int temp = 1;
fp = fopen("input.txt","r");

if(fp == NULL)
{
   printf("File not found");
   return 0;
}

fgets(input,100,fp);
fclose(fp);

printf("Input Expression: %s\n",input);
printf("\nThree Address Code:\n");

char op1, op2, operator;

/* assuming format like a=b+c*d-e */
char result = input[0];

op1 = input[2];
operator = input[3];
op2 = input[4];

printf("t%d = %c %c %c\n",temp,op1,operator,op2);

for(i=5;i<strlen(input);i=i+2)
{
   op1 = 't';
   operator = input[i];
   op2 = input[i+1];

   printf("t%d = t%d %c %c\n",temp+1,temp,operator,op2);
   temp++;
}

printf("%c = t%d\n",result,temp);

return 0;
}
```

13. Write a program to simulate PDA for language having strings with equal number of a's and b's.
Write a program to simulate TM for language L = { a^n b^n c^n | n ≥ 0 }.

PDA:

```
#include <stdio.h>
#include <string.h>
#define MAX 100
char stack[MAX];
int top = -1;
/* push operation */
void push(char x)
{
stack[++top] = x;
}
/* pop operation */
void pop()
{
if(top != -1)
top--;
}
int main()
{
char input[100];
int i;
printf("Enter string (a and b only): ");
scanf("%s", input);

for(i = 0; i < strlen(input); i++)
{
   if(input[i] == 'a')
   {
      if(top != -1 && stack[top] == 'b')
         pop();
      else
         push('a');
   }
   else if(input[i] == 'b')
   {
      if(top != -1 && stack[top] == 'a')
         pop();
      else
         push('b');
   }
   else
   {
      printf("Invalid symbol\n");
      return 0;
   }
}

if(top == -1)
```

```
    printf("Accepted (Equal number of a's and b's)\n");
else
    printf("Rejected\n");

return 0;
}
```

TM:

```
#include <stdio.h>
#include <string.h>
int main()
{
char tape[100];
int i, j, k;
int len;
printf("Enter string: ");
scanf("%s", tape);

len = strlen(tape);

while(1)
{
    /* Step 1: find first 'a' */
    for(i = 0; i < len; i++)
    {
        if(tape[i] == 'a')
        {
            tape[i] = 'X';
            break;
        }
    }

    /* If no 'a' found, stop */
    if(i == len)
        break;

    /* Step 2: find 'b' */
    for(j = i + 1; j < len; j++)
    {
        if(tape[j] == 'b')
        {
            tape[j] = 'Y';
            break;
        }
    }

    if(j == len)
    {
        printf("Rejected\n");
        return 0;
    }

    /* Step 3: find 'c' */
```

```
    for(k = j + 1; k < len; k++)
    {
        if(tape[k] == 'c')
        {
            tape[k] = 'Z';
            break;
        }
    }

    if(k == len)
    {
        printf("Rejected\n");
        return 0;
    }
}

/* Final check */
for(i = 0; i < len; i++)
{
    if(tape[i] != 'X' && tape[i] != 'Y' && tape[i] != 'Z')
    {
        printf("Rejected\n");
        return 0;
    }
}

printf("Accepted (String belongs to a^n b^n c^n)\n");

return 0;
}
```